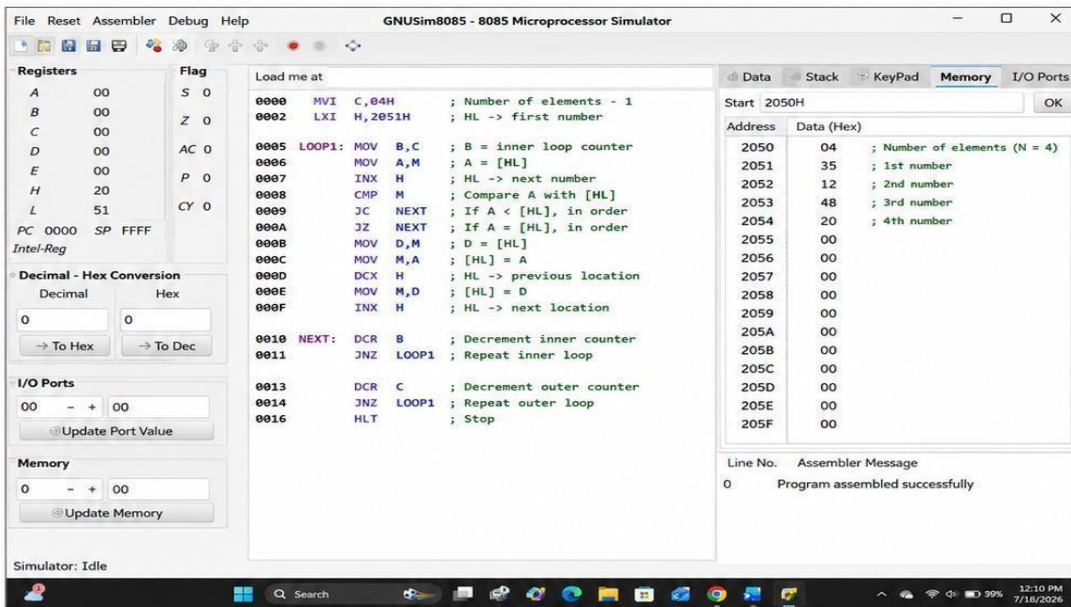


Name: K.MAHESH BABU

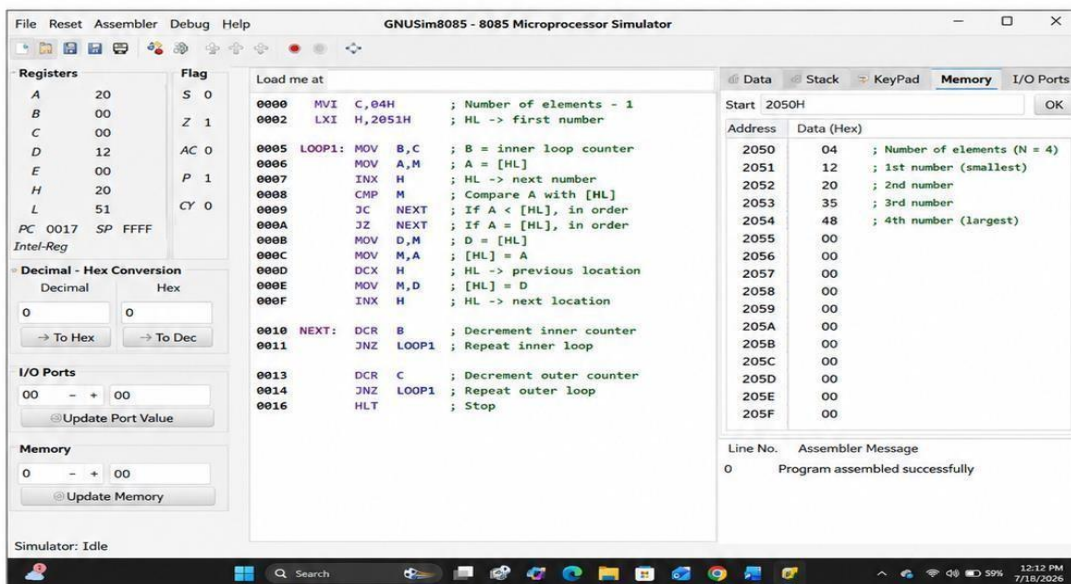
Regno :192525122

16. Write an assembly language program to arrange numbers in Ascending order.

Program 16: Arrange Numbers in Ascending Order (8085)



Before Execution (Unsorted Numbers)



After Execution (Sorted Numbers in Ascending Order)

Observation:

Before execution, the numbers were 35H, 12H, 48H, 20H.

After execution, the numbers are arranged in ascending order as 12H, 20H, 35H, 48H.

17. Write an assembly language program to arrange numbers in Descending order.

17. Arrange Numbers in Descending Order (8085)

Program : To arrange numbers in Descending order.

Before Execution (Unsorted Numbers)

Registers: A 00, B 00, C 00, D 00, E 00, H 20, L 51, PC 0000, SP FFFF, Int-Reg 0. Flag: S 0, Z 0, AC 0, P 0, CY 0.

Decimal - Hex Conversion: 0 To Hex, 0 To Dec.

I/O Ports: 00 Update Port Value.

Memory: 00 Update Memory.

Load me at:

```

0000 MVI C,04H ; Number of elements - 1
0002 LXI H,2051H ; HL -> first number
0005 LOOP1: MOV B,C ; B = inner loop counter
0006 MOV A,M ; A = [HL]
0007 INX H ; HL -> next number
0008 CMP H ; Compare A with [HL]
0009 JNC NEXT ; If A >= [HL], in order
000A MOV D,M ; D = [HL]
000B MOV M,A ; [HL] = A
000C DCX H ; HL -> previous location
000D MOV M,D ; [HL] = D
000E INX H ; HL -> next location
0010 NEXT: DCR B ; Decrement inner counter
0011 JNZ LOOP2 ; Repeat inner loop
0013 DCR C ; Decrement outer counter
0014 JNZ LOOP1 ; Repeat outer loop
0016 HLT ; Stop

```

Memory Table:

Address	Data (Hex)	Comment
2050	04	; Number of elements (N = 4)
2051	35	; 1st number
2052	12	; 2nd number
2053	48	; 3rd number
2054	20	; 4th number
2055	00	
2056	00	
2057	00	
2058	00	

Line No. Assembler Message
0 Program assembled successfully

After Execution (Sorted Numbers in Descending Order)

Registers: A 48, B 00, C 00, D 20, E 00, H 20, L 55, PC 0017, SP FFFF, Int-Reg 0. Flag: S 0, Z 0, AC 0, P 0, CY 1.

Decimal - Hex Conversion: 0 To Hex, 0 To Dec.

I/O Ports: 00 Update Port Value.

Memory: 00 Update Memory.

Load me at:

```

0000 MVI C,04H ; Number of elements - 1
0002 LXI H,2051H ; HL -> first number
0005 LOOP1: MOV B,C ; B = inner loop counter
0006 MOV A,M ; A = [HL]
0007 INX H ; HL -> next number
0008 CMP H ; Compare A with [HL]
0009 JNC NEXT ; If A >= [HL], in order
000A MOV D,M ; D = [HL]
000B MOV M,A ; [HL] = A
000C DCX H ; HL -> previous location
000D MOV M,D ; [HL] = D
000E INX H ; HL -> next location
0010 NEXT: DCR B ; Decrement inner counter
0011 JNZ LOOP2 ; Repeat inner loop
0013 DCR C ; Decrement outer counter
0014 JNZ LOOP1 ; Repeat outer loop
0016 HLT ; Stop

```

Memory Table:

Address	Data (Hex)	Comment
2050	04	; Number of elements (N = 4)
2051	48	; 1st number (largest)
2052	35	; 2nd number
2053	20	; 3rd number
2054	12	; 4th number (smallest)
2055	00	
2056	00	
2057	00	
2058	00	

Line No. Assembler Message
0 Program assembled successfully

Observation:

Before execution, the numbers were : 35H, 12H, 48H, 20H.

After execution, the numbers are arranged in Descending order as : 48H, 35H, 20H, 12H.

18. Write an assembly language program to find the largest number in an array.

18. Write an assembly language program to find the largest number in an array (8085).

Before Execution

After Execution

Input (Before Execution) :
 Number of elements (N) = 5
 Array elements (starting from 2051H) = 13H, 2FH, 07H, 8AH, 3EH

Output (After Execution) :
 Largest number = 8AH (stored at memory location 2060H)

19. Write an assembly language program to find the Minimum number in an array.

19. Write an Assembly Language Program to Find the Minimum Number in an Array (8085).

Before Execution (Unsorted Array)

Registers: A 00, B 00, C 00, D 00, E 00, H 20, L 51, PC 2000, SP FFFF, Int-Reg 00. Flag: S 0, Z 0, AC 0, P 0, CY 0.

Decimal - Hex Conversion: 0 → To Hex, 0 → To Dec.

I/O Ports: 00 - + 00, Update Port Value.

Memory: 0 - + 00, Update Memory.

Load me at:

```

2000 MVI C,05H ; Number of elements (N = 5)
2002 LXI H,2051H ; HL -> first element
2005 MOV A,M ; A <- first element (assume minimum)
2006 INX H ; HL -> next element
2007 DCR C ; Decrement count (N-1)

2008 LOOP: CMP ; Compare A with [HL]
2009 JNC NEXT ; If A <= [HL], no change
200B MOV A,M ; Else, A <- [HL] (new minimum)
200C NEXT: INX H ; HL -> next element
200D DCR C ; Decrement count
200E JNZ LOOP ; Repeat until all compared
2010 STA 2060H ; Store minimum at 2060H
2013 HLT ; Stop

```

Memory Table:

Address	Data (Hex)	Comment
2050	05	; Number of elements (N = 5)
2051	13	; 1st element
2052	2F	; 2nd element
2053	07	; 3rd element
2054	8A	; 4th element
2055	3E	; 5th element
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	--	; Location to store minimum

Line No. 0 Assembler Message: Program assembled successfully

After Execution (Minimum Number Stored)

Registers: A 07, B 00, C 00, D 00, E 00, H 20, L 56, PC 2014, SP FFFF, Int-Reg 00. Flag: S 0, Z 0, AC 0, P 0, CY 0.

Decimal - Hex Conversion: 0 → To Hex, 0 → To Dec.

I/O Ports: 00 - + 00, Update Port Value.

Memory: 0 - + 00, Update Memory.

Load me at:

```

2000 MVI C,05H ; Number of elements (N = 5)
2002 LXI H,2051H ; HL -> first element
2005 MOV A,M ; A <- first element (assume minimum)
2006 INX H ; HL -> next element
2007 DCR C ; Decrement count (N-1)

2008 LOOP: CMP ; Compare A with [HL]
2009 JNC NEXT ; If A <= [HL], no change
200B MOV A,M ; Else, A <- [HL] (new minimum)
200C NEXT: INX H ; HL -> next element
200D DCR C ; Decrement count
200E JNZ LOOP ; Repeat until all compared
2010 STA 2060H ; Store minimum at 2060H
2013 HLT ; Stop

```

Memory Table:

Address	Data (Hex)	Comment
2050	05	; Number of elements (N = 5)
2051	13	; 1st element
2052	2F	; 2nd element
2053	07	; 3rd element
2054	8A	; 4th element
2055	3E	; 5th element
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	07	; Minimum number stored here

Line No. 0 Assembler Message: Program assembled successfully

Input (Before Execution) :
Number of elements (N) = 5
Array elements (starting from 2051H) = 13H, 2FH, 07H, 8AH, 3EH

Output (After Execution) :
Minimum number = 07H (stored at memory location 2060H)

20. Write an assembly language program to find the LCM of 2 numbers

20. Write an Assembly Language Program to Find the LCM of Two Numbers (8085).

Before Execution (Input)

The simulator window shows the following assembly code:

```

2000 MVI C,02H ; First number = 02H (2)
2002 MVI D,03H ; Second number = 03H (3)
2004 MOV A,C ; A = first number
2005 MOV B,D ; B = second number
2006 DAD B ; HL = A + B
2007 MOV H,D ; H = second number
2008 MOV L,C ; L = first number
2009 CMP L ; Compare H and L
200A JZ STORE ; If equal, GCD found
200B JC SMALL ; If H < L, go to SMALL
200C SUB L ; H = H - L
200D JMP LOOP
200E SMALL: CMP H ; Compare L and H
200F JZ STG ; If L < H, go to BIG
2010 JC BIGB ; L = L - H
2011 SUB H ; H = H - L
2012 JMP SMALL
2013 BIG: SUB L ; GCD in H
2014 JMP BIG
2015 STORE: MOV E,H ; L = GCD
2016 MVI D,00H ; H=0
2018 MOV A,C ; HL = (First x Second)
2019 MOV L,E ; Divide by GCD
201A MOV H,D
201B DAD B
201C MOV B,L
201D MOV C,H
201E MOV A,B ; Loop for division
201F DCR A
2020 MOV B,A
2021 DIVI: MOV A,C ; Subtract GCD
2022 SBB L
2023 MOV C,A
2024 MOV A,B ; Check if quotient = 0
2025 ORA A
2026 JZ DONE
2027 DCR B
2028 JMP DIVI
2029 DONE: MOV L,L ; L = LCM
202A MOV H,H
202C STA 2050H ; Store LCM at 2050H
202F HLT
  
```

The memory window shows the LCM value 06H stored at address 2050H.

After Execution (LCM Stored)

The simulator window shows the following assembly code:

```

2000 MVI C,02H ; First number = 02H (2)
2002 MVI D,03H ; Second number = 03H (3)
2004 MOV A,C ; A = first number
2005 MOV B,D ; B = second number
2006 DAD B ; HL = A + B
2007 MOV H,D ; H = second number
2008 MOV L,C ; L = first number
2009 CMP L ; Compare H and L
200A JZ STORE ; If equal, GCD found
200B JC SMALL ; If H < L, go to SMALL
200C SUB L ; H = H - L
200D JMP LOOP
200E SMALL: CMP H ; Compare L and H
200F JZ STG ; If L < H, go to BIG
2010 JC BIGB ; L = L - H
2011 SUB H ; H = H - L
2012 JMP SMALL
2013 BIG: SUB L ; GCD in H
2014 JMP BIG
2015 STORE: MOV E,H ; L = GCD
2016 MVI D,00H ; H=0
2018 MOV A,C ; HL = (First x Second)
2019 MOV L,E ; Divide by GCD
201A MOV H,D
201B DAD B
201C MOV B,L
201D MOV C,H
201E MOV A,B ; Loop for division
201F DCR A
2020 MOV B,A
2021 DIVI: MOV A,C ; Subtract GCD
2022 SBB L
2023 MOV C,A
  
```

The memory window shows the LCM value 06H stored at address 2050H.

Input (Before Execution) : First Number = 02H (2), Second Number = 03H (3)

Output (After Execution) : LCM = 06H (6)

LCM is stored at memory location 2050H.

21. Write an assembly language program to find the GCD of 2 numbers.

21. Write an Assembly Language Program to Find the GCD of Two Numbers (8085).

Before Execution (Input)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers

A	00
B	00
C	00
D	00
E	00
H	20
L	50
PC	2000
SP	FFFF
Int-Reg	00

Flag

S	0
Z	0
AC	0
P	0
CY	0

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

00 - + 00

Update Port Value

Memory

0 - + 00

Update Memory

Simulator: Idle

Load me at

```

2000 MVI B,15H ; First number = 15H (21)
2002 MVI C,09H ; Second number = 09H (9)
2004 MOV A,B ; A = First number
2005 MOV D,C ; D = Second number
2006 LOOP: CMP D ; Compare A and D
2007 JZ DONE ; If equal, GCD found
2009 JC L1 ; If A < D, go to L1
200A SUB D ; A = A - D
200B JMP LOOP ; Repeat loop
200D L1: MOV A,D ; D = D - B
200E SUB B ; D = D - B
200F MOV D,A ; Update D
2010 JMP LOOP
2013 DONE: MOV E,A ; GCD in A -> store in E
2014 STA 2050H ; Store GCD at 2050H
2017 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	00	; Location to store GCD
2051	15	; First number = 15H (21)
2052	09	; Second number = 09H (9)
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	--	

Line No. Assembler Message

0 Program assembled successfully

After Execution (GCD Stored)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers

A	03
B	15
C	09
D	03
E	03
H	20
L	50
PC	2018
SP	FFFF
Int-Reg	00

Flag

S	0
Z	1
AC	0
P	1
CY	0

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

00 - + 00

Update Port Value

Memory

0 - + 00

Update Memory

Simulator: Idle

Load me at

```

2000 MVI B,15H ; First number = 15H (21)
2002 MVI C,09H ; Second number = 09H (9)
2004 MOV A,B ; A = First number
2005 MOV D,C ; D = Second number
2006 LOOP: CMP D ; Compare A and D
2007 JZ DONE ; If equal, GCD found
2009 JC L1 ; If A < D, go to L1
200A SUB D ; A = A - D
200B JMP LOOP ; Repeat loop
200D L1: MOV A,D ; D = D - B
200E SUB B ; D = D - B
200F MOV D,A ; Update D
2010 JMP LOOP
2013 DONE: MOV E,A ; GCD in A -> store in E
2014 STA 2050H ; Store GCD at 2050H
2017 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	03	; GCD = 03H (3)
2051	15	; First number = 15H (21)
2052	09	; Second number = 09H (9)
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	--	

Line No. Assembler Message

0 Program assembled successfully

Input (Before Execution) : First Number = 15H (21), Second Number = 09H (9)

Output (After Execution) : GCD = 03H (3)

GCD is stored at memory location 2050H.

22. Write an assembly language program to find factorial of n in the given number.

22. Write an Assembly Language Program to Find Factorial of N in the Given Number (8085).

Before Execution (Input)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers: A 00, B 00, C 00, D 00, E 00, H 20, L 50, PC 2000, SP FFFF, Int-Reg 00. Flag: S 0, Z 0, AC 0, P 0, CY 0.

Decimal - Hex Conversion: 0 to 0.

I/O Ports: 00 to 00.

Memory: 0 to 00.

Load me at:

```

2000 MVI C,05H ; Input N = 05 (5)
2002 MVI B,01H ; Initialize result = 01
2004 MOV A,C ; A = N
2006 LOOP:ORA A ; Check if A = 0
2007 JZ DONE ; If yes, go to DONE
2009 MOV DB ; D = Result
200A MOV E,A ; E = N
200B CALL MULT ; B = B * A
200E DCR A ; N = N - 1
200F JMP LOOP ; Repeat loop
2011 MULT:MOV HB ; Multiply B by E (8-bit x 8-bit)
2012 MVI L,00H ; HL = 0000H (16-bit result)
2014 MVI: MOV AL ;
2015 ADD B ; L = L + B
2016 MOV L,A ;
2017 JNC SKIP ;
2018 INR H ; If carry, increment H
2019 SKIP:DCR E ; E = E - 1
201A JNZ M1 ; Repeat until E = 0
201C MOV B,L ; Store low byte in B
201D MOV C,H ; Store high byte in C
201E RET ;
201F DONE:MOV A ; Low byte of result in A
2020 STA 2051H ; Store low byte at 2051H
2023 MOV A,C ; High byte in A
2024 STA 2052H ; Store high byte at 2052H
2027 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	05	; N (Input) = 05 (5)
2051	00	; Low byte of factorial
2052	00	; High byte of factorial
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	--	

Line No. Assembler Message
0 Program assembled successfully

Simulator: Idle

After Execution (Factorial Stored)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers: A 00, B 78, C 00, D 00, E 00, H 20, L 78, PC 2027, SP FFFF, Int-Reg 00. Flag: S 0, Z 1, AC 0, P 1, CY 0.

Decimal - Hex Conversion: 0 to 0.

I/O Ports: 00 to 00.

Memory: 0 to 00.

Load me at:

```

2000 MVI C,05H ; Input N = 05 (5)
2002 MVI B,01H ; Initialize result = 01
2004 MOV A,C ; A = N
2006 LOOP:ORA A ; Check if A = 0
2007 JZ DONE ; If yes, go to DONE
2009 MOV DB ; D = Result
200A MOV E,A ; E = N
200B CALL MULT ; B = B * A
200E DCR A ; N = N - 1
200F JMP LOOP ; Repeat loop
2011 MULT:MOV HB ; Multiply B by E (8-bit x 8-bit)
2012 MVI L,00H ; HL = 0000H (16-bit result)
2014 MVI: MOV AL ;
2015 ADD B ; L = L + B
2016 MOV L,A ;
2017 JNC SKIP ;
2018 INR H ; If carry, increment H
2019 SKIP:DCR E ; E = E - 1
201A JNZ M1 ; Repeat until E = 0
201C MOV B,L ; Store low byte in B
201D MOV C,H ; Store high byte in C
201E RET ;
201F DONE:MOV A ; Low byte of result in A
2020 STA 2051H ; Store low byte at 2051H
2023 MOV A,C ; High byte in A
2024 STA 2052H ; Store high byte at 2052H
2027 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	05	; N (Input) = 05 (5)
2051	78	; Low byte of factorial
2052	00	; High byte of factorial
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	--	

Line No. Assembler Message
0 Program assembled successfully

Simulator: Idle

Input (Before Execution) : N = 05 (5)
Output (After Execution) : Factorial of 5 = 120 = 0078H
Result stored at memory location 2051H (Low byte) and 2052H (High byte).

23. Write a program to convert Decimal number to Hexadecimal number.

23. Write a program to convert Decimal number to Hexadecimal number (8085).

Before Execution (Input)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers

Register	Value	Flag	Value
A	00	S	0
B	00	Z	0
C	00	AC	0
D	00	P	0
E	00	CY	0
H	00		
L	00		
PC	2000	SP	FFFF
Int-Reg	00		

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

Input: 00 Output: 00

Memory

Input: 0 Output: 00

Simulator: Idle

Load me at

```

2000 MVI A,4DH ; Load decimal number (example : 77)
2002 MVI B,00H ; B will hold high nibble (MSN)
2004 MVI C,00H ; C will hold low nibble (LSN)
2006 MOV B,A ; Copy number to B
2007 RRC ; Rotate right 4 times to get high nibble
2008 RRC
2009 RRC
200A RRC
200B ANI 0FH ; Mask lower 4 bits (High nibble)
200D STA 2050H ; Store high nibble at 2050H
2010 MOV A,B ; Get original number back
2011 MOV A,4BH ; (Reload original for safety)
2013 ANI 0FH ; Mask higher 4 bits (Low nibble)
2015 STA 2051H ; Store low nibble at 2051H
2018 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	00	; High nibble will be stored
2051	00	; Low nibble will be stored
2052	00	
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	--	

Line No. Assembler Message

0 Program assembled successfully

After Execution (Hexadecimal Result Stored)

File Reset Assembler Debug Help

GNUSim8085 - 8085 Microprocessor Simulator

Registers

Register	Value	Flag	Value
A	00	S	0
B	00	Z	0
C	00	AC	0
D	00	P	1
E	00	CY	0
H	00		
L	00		
PC	2018	SP	FFFF
Int-Reg	00		

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

Input: 00 Output: 00

Memory

Input: 0 Output: 00

Simulator: Idle

Load me at

```

2000 MVI A,4DH ; Load decimal number (example : 77)
2002 MVI B,00H ; B will hold high nibble (MSN)
2004 MVI C,00H ; C will hold low nibble (LSN)
2006 MOV B,A ; Copy number to B
2007 RRC ; Rotate right 4 times to get high nibble
2008 RRC
2009 RRC
200A RRC
200B ANI 0FH ; Mask lower 4 bits (High nibble)
200D STA 2050H ; Store high nibble at 2050H
2010 MOV A,B ; Get original number back
2011 MOV A,4BH ; (Reload original for safety)
2013 ANI 0FH ; Mask higher 4 bits (Low nibble)
2015 STA 2051H ; Store low nibble at 2051H
2018 HLT ; Stop

```

Start 2050H

Address	Data (Hex)	Comment
2050	04	; High nibble (MSN) = 4
2051	0D	; Low nibble (LSN) = D
2052	00	
2053	00	
2054	00	
2055	00	
2056	00	
2057	00	
2058	00	
2059	00	
205A	00	
205B	00	
205C	00	
205D	00	
205E	00	
205F	00	
2060	--	

Line No. Assembler Message

0 Program assembled successfully

Input (Decimal number) : 77 (4DH)

Output (Hexadecimal) : 4D (High nibble = 04H, Low nibble = 0DH)

Result stored at 2050H = 04H and 2051H = 0DH.